

Lab4

组员：杨桑多杰，张磊，徐南海

练习0：填写已有实验

本实验依赖实验2/3。请把你做的实验2/3的代码填入本实验中代码中有“LAB2”,“LAB3”的注释相应部分。

练习1：分配并初始化一个进程控制块（需要编码）

`alloc_proc`函数（位于`kern/process/proc.c`中）负责分配并返回一个新的`struct proc_struct`结构，用于存储新建立的内核线程的管理信息。`ucore`需要对这个结构进行最基本的初始化，你需要完成这个初始化过程。

请在实验报告中简要说明你的设计实现过程。请回答如下问题：

- 请说明`proc_struct`中 `struct context context` 和 `struct trapframe *tf` 成员变量含义和在本实验中的作用是啥？（提示通过看代码和编程调试可以判断出来）

解答

设计思路

首先定位到 `struct proc_struct` 结构位于 `kern/process/proc.h` 中,代码如下:

```
struct proc_struct
{
    enum proc_state state;           // Process state
    int pid;                         // Process ID
    int runs;                        // the running times of Proces
    uintptr_t kstack;                // Process kernel stack
    volatile bool need_resched;      // bool value: need to be rescheduled to release CPU?
    struct proc_struct *parent;      // the parent process
    struct mm_struct *mm;            // Process's memory management field
    struct context context;          // Switch here to run process
    struct trapframe *tf;            // Trap frame for current interrupt
    uintptr_t pgdir;                 // the base addr of Page Directroy Table(PDT)
    uint32_t flags;                  // Process flag
    char name[PROC_NAME_LEN + 1];    // Process name
    list_entry_t list_link;          // Process link list
    list_entry_t hash_link;          // Process hash list
};
```

按照实验手册所编写代码如下:

```
// alloc_proc - alloc a proc_struct and init all fields of proc_struct
static struct proc_struct *
alloc_proc(void)
```

```

{
    struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
    if (proc != NULL)
    {
        // LAB4:EXERCISE1
        /*
         * below fields in proc_struct need to be initialized
         *
         * enum proc_state state;           // Process state
         * int pid;                         // Process ID
         * int runs;                        // the running times of Proces
         * uintptr_t kstack;                // Process kernel stack
         * volatile bool need_resched;      // bool value: need to be
rescheduled to release CPU?
         * struct proc_struct *parent;      // the parent process
         * struct mm_struct *mm;            // Process's memory management
field
         * struct context context;          // Switch here to run process
         * struct trapframe *tf;            // Trap frame for current
interrupt
         * uintptr_t pgdir;                 // the base addr of Page
Directory Table(PDT)
         * uint32_t flags;                  // Process flag
         * char name[PROC_NAME_LEN + 1];    // Process name
        */
        proc->state = PROC_UNINIT;
        proc->pid = -1;
        proc->runs = 0;
        proc->kstack = 0;
        proc->need_resched = 0;
        proc->parent = NULL;
        proc->mm = NULL;
        memset(&(proc->context), 0, sizeof(struct context));
        proc->tf = NULL;
        proc->pgdir = boot_pgdir_pa;
        proc->flags = 0;
        memset(proc->name, 0, PROC_NAME_LEN);
        list_init(&(proc->list_link));
        list_init(&(proc->hash_link));

    }
    return proc;
}

```

实现内核线程首先需要给线程创建一个进程，于是我们需要给进程控制块指针（`struct proc_struct* proc`）初始化分配内存空间，而进程控制块指针中包含如下变量：

- state: 进程状态，`proc.h`中定义了四种状态：创建（UNINIT）、睡眠（SLEEPING）、就绪（RUNNABLE）、退出（ZOMBIE，等待父进程回收其资源）
- pid: 进程ID，调用本函数时尚未指定，默认值设为-1
- runs: 线程运行总数，默认值0

- kstack: 内核栈地址,该进程分配的地址为0, 因为还没有执行, 也没有被重定位, 因为默认地址都是从0开始的
- need_resched: 标志位, 表示该进程是否需要重新参与调度以释放CPU, 初值0 (false, 表示不需要)
- parent: 父进程控制块指针, 初值NULL
- mm: 用户进程虚拟内存管理单元指针, 由于系统进程没有虚存, 其值为NULL
- context: 进程上下文, 默认值全零
- tf: 中断帧指针, 默认值NULL
- pgdir: 即页目录 (Page Directory) 的基址, 表明由于该内核线程在内核中运行, 故采用为uCore内核已经建立的页表, 即设置为在uCore内核页表的起始地址boot_pgdir
- flags: 进程标志位, 默认值0
- name: 进程名数组,初始化为0
- 初始化 list_link 与 hash_link

通过以上的代码, 我们就可以完成PCB的分配和初始化。

问题回答

- `struct context context`: context是保存进程执行的上下文, 用于保存进程切换时父进程的一些寄存器值 (`ra;sp;s0;s1;s2;s3; s4; s5;s6;s7;s8;s9;s10;s11`)。可用于在进程切换中还原之前的运行状态。在通过 `proc_run` 切换到CPU上运行时, 需要调用 `switch_to` 将原进程的寄存器保存, 以便下次切换回去时读出, 保持之前的状态。

`struct context` 的结构在 `kern/process/proc.h` 中的定义如下:

```

struct context
{
    uintptr_t ra; //ra: 返回地址寄存器, 用于保存函数调用后的返回地址。
    uintptr_t sp; //sp: 栈指针寄存器, 指向当前线程的栈顶。

    //s0 到 s11: 这些是保存寄存器 (scratch registers), 用于保存临时数据, 它们在函数调用时不需要被保存和
    恢复, 因为它们不会被调用者所保留。
    uintptr_t s0;
    uintptr_t s1;
    uintptr_t s2;
    uintptr_t s3;
    uintptr_t s4;
    uintptr_t s5;
    uintptr_t s6;
    uintptr_t s7;
    uintptr_t s8;
    uintptr_t s9;
    uintptr_t s10;
    uintptr_t s11;
};

```

为什么我们不需要保存所有的寄存器呢？这里我们巧妙地利用了编译器对于函数的处理。我们知道寄存器可以分为调用者保存（*caller-saved*）寄存器和被调用者保存（*callee-saved*）寄存器。因为线程切换在一个函数当中，所以编译器会自动帮助我们生成保存和恢复调用者保存寄存器的代码，在实际的进程切换过程中我们只需要保存被调用者保存寄存器就好了。

- `struct trapframe *tf`：`tf`是中断帧的指针，总是指向内核栈的某个位置：当进程从用户态转移到内核态时，中断帧`tf`记录了进程在被中断前的状态，比如部分寄存器的值。当内核需要跳回用户态时，需要调整中断帧以恢复让进程继续执行的各寄存器值。

`struct trapframe *tf` 所这涉及寄存器如下：

```
struct pushregs
{
    uintptr_t zero; // Hard-wired zero
    uintptr_t ra;   // Return address
    uintptr_t sp;   // Stack pointer
    uintptr_t gp;   // Global pointer
    uintptr_t tp;   // Thread pointer
    uintptr_t t0;   // Temporary
    uintptr_t t1;   // Temporary
    uintptr_t t2;   // Temporary
    uintptr_t s0;   // Saved register/frame pointer
    uintptr_t s1;   // Saved register
    uintptr_t a0;   // Function argument/return value
    uintptr_t a1;   // Function argument/return value
    uintptr_t a2;   // Function argument
    uintptr_t a3;   // Function argument
    uintptr_t a4;   // Function argument
    uintptr_t a5;   // Function argument
    uintptr_t a6;   // Function argument
    uintptr_t a7;   // Function argument
    uintptr_t s2;   // Saved register
    uintptr_t s3;   // Saved register
    uintptr_t s4;   // Saved register
    uintptr_t s5;   // Saved register
    uintptr_t s6;   // Saved register
    uintptr_t s7;   // Saved register
    uintptr_t s8;   // Saved register
    uintptr_t s9;   // Saved register
    uintptr_t s10;  // Saved register
    uintptr_t s11;  // Saved register
    uintptr_t t3;   // Temporary
    uintptr_t t4;   // Temporary
    uintptr_t t5;   // Temporary
    uintptr_t t6;   // Temporary
};

struct trapframe
{
    struct pushregs gpr; //32个通用寄存器
```

```
//Lab3 trap中所使用的处理异常的寄存器
uintptr_t status; //状态
uintptr_t epc; //被中断指令的虚拟地址
uintptr_t badvaddr; //存放导致失败的访存地址
uintptr_t cause; //中断或异常的具体原因
};
```

练习2：为新创建的内核线程分配资源（需要编码）

创建一个内核线程需要分配和设置好很多资源。kernel_thread函数通过调用do_fork函数完成具体内核线程的创建工作。do_kernel函数会调用alloc_proc函数来分配并初始化一个进程控制块，但alloc_proc只是找到了一小块内存用以记录进程的必要信息，并没有实际分配这些资源。ucore一般通过do_fork实际创建新的内核线程。do_fork的作用是，创建当前内核线程的一个副本，它们的执行上下文、代码、数据都一样，但是存储位置不同。因此，我们实际需要“fork”的东西就是stack和trapframe。在这个过程中，需要给新内核线程分配资源，并且复制原进程的状态。你需要完成在kern/process/proc.c中的do_fork函数中的处理过程。它的大致执行步骤包括：

- 调用alloc_proc，首先获得一块用户信息块。
- 为进程分配一个内核栈。
- 复制原进程的内存管理信息到新进程（但内核线程不必做此事）
- 复制原进程上下文到新进程
- 将新进程添加到进程列表
- 唤醒新进程
- 返回新进程号

请在实验报告中简要说明你的设计实现过程。请回答如下问题：

- 请说明ucore是否做到给每个新fork的线程一个唯一的id？请说明你的分析和理由。

解答

设计思路

代码如下：

```
int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
{
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;
    if (nr_process >= MAX_PROCESS)
    {
        goto fork_out;
    }
    ret = -E_NO_MEM;
    // LAB4:EXERCISE2
    /*
     * Some Useful MACROS, Functions and DEFINES, you can use them in below implementation.
```

```

* MACROS or Functions:
*   alloc_proc:   create a proc struct and init fields (lab4:exercise1)
*   setup_kstack: alloc pages with size KSTACKPAGE as process kernel stack
*   copy_mm:      process "proc" duplicate OR share process "current"'s mm according
clone_flags
*               if clone_flags & CLONE_VM, then "share" ; else "duplicate"
*   copy_thread:  setup the trapframe on the process's kernel stack top and
*               setup the kernel entry point and stack of process
*   hash_proc:    add proc into proc hash_list
*   get_pid:      alloc a unique pid for process
*   wakeup_proc:  set proc->state = PROC_RUNNABLE
* VARIABLES:
*   proc_list:    the process set's list
*   nr_process:   the number of process set
*/

//   1. call alloc_proc to allocate a proc_struct
//   2. call setup_kstack to allocate a kernel stack for child process
//   3. call copy_mm to dup OR share mm according clone_flag
//   4. call copy_thread to setup tf & context in proc_struct
//   5. insert proc_struct into hash_list && proc_list
//   6. call wakeup_proc to make the new child process RUNNABLE
//   7. set ret vaule using child proc's pid
proc = alloc_proc();
if (proc == NULL)
{
    goto fork_out;
}
proc->parent = current;
if (setup_kstack(proc) != 0)
{
    goto bad_fork_cleanup_proc;
}
if (copy_mm(clone_flags, proc) != 0)
{
    goto bad_fork_cleanup_kstack;
}
copy_thread(proc, stack, tf);
proc->pid = get_pid();
hash_proc(proc);
list_add(&proc_list, &(proc->list_link));
nr_process++;
wakeup_proc(proc);
ret = proc->pid;

fork_out:
    return ret;

bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;

```

```
}
```

- 首先调用 `alloc_proc` 为 `proc` 分配一个进程。如果分配失败，那么直接返回“没有空闲进程”的错误代码。
- 将当前子进程的 `parent` 指针指向 `current` 进程。
- 然后调用 `setup_kstack` 分配内核栈。如果分配失败，那么首先把刚刚分配的 `proc` 释放掉，然后再返回“没有空闲进程”的错误代码。
- 接着调用 `copy_mm` 来复制 `mm_struct`（内核线程情况下为“无动作”返回 0）。
- 调用 `copy_thread` 复制线程初始上下文和 `trapframe`。
- 将新进程添加到进程列表，分配唯一 `pid`，加入哈希表与进程双向链表，增加 `nr_process`。
- 可以将 `proc` 的状态设为 `PROC_RUNNABLE`
- 最后返回 `proc` 的 `pid`。

问题回答

ucore 能为每个新 fork 的线程分配唯一的 pid。理由如下：

`get_pid()` 函数定义如下：

```
static int
get_pid(void)
{
    static_assert(MAX_PID > MAX_PROCESS);
    struct proc_struct *proc;
    list_entry_t *list = &proc_list, *le;
    static int next_safe = MAX_PID, last_pid = MAX_PID;
    if (++last_pid >= MAX_PID)
    {
        last_pid = 1;
        goto inside;
    }
    if (last_pid >= next_safe)
    {
        inside:
        next_safe = MAX_PID;
        repeat:
        le = list;
        while ((le = list_next(le)) != list)
        {
            proc = le2proc(le, list_link);
            if (proc->pid == last_pid)
            {
                if (++last_pid >= next_safe)
                {
                    if (last_pid >= MAX_PID)
                    {
                        last_pid = 1;
                    }
                    next_safe = MAX_PID;
                }
            }
        }
    }
}
```

```

        goto repeat;
    }
}
else if (proc->pid > last_pid && next_safe > proc->pid)
{
    next_safe = proc->pid;
}
}
}
return last_pid;
}

```

- 使用 `get_pid()` 线性分配且带“避冲”逻辑：在 `proc_list` 上遍历，若遇已占用 `pid`，会前移并记录下一个安全上界 `next_safe`，直到找到未被占用的 `pid`；溢出时从 1 重新循环，且范围受限于 `0 < pid < MAX_PID`。
- `find_proc` 和 `hash_proc` 通过 `pid_hashfn` 放入 `hash_list`，且 `find_proc` 只返回精确匹配的 `pid`。
- `nr_process < MAX_PROCESS` 且 `static_assert(MAX_PID > MAX_PROCESS)`，保证有足够 `pid` 空间避免因容量导致的重复冲突。
- 在 `do_fork` 中，先获得 `pid`，再插入全局结构（哈希表和进程链表），在任意失败前未暴露重复 `pid`，确保可见集合中的 `pid` 唯一。

练习3：编写proc_run 函数（需要编码）

`proc_run`用于将指定的进程切换到CPU上运行。它的大致执行步骤包括：

- 检查要切换的进程是否与当前正在运行的进程相同，如果相同则不需要切换。
- 禁用中断。你可以使用 `/kern/sync/sync.h` 中定义好的宏 `local_intr_save(x)` 和 `local_intr_restore(x)` 来实现关、开中断。
- 切换当前进程为要运行的进程。
- 切换页表，以便使用新进程的地址空间。`/libs/riscv.h` 中提供了 `lsatp(unsigned int pgdir)` 函数，可实现修改SATP寄存器值的功能。
- 实现上下文切换。`/kern/process` 中已经预先编写好了 `switch.S`，其中定义了 `switch_to()` 函数。可实现两个进程的context切换。
- 允许中断。

请回答如下问题：

- 在本实验的执行过程中，创建且运行了几个内核线程？

完成代码编写后，编译并运行代码：`make qemu`

解答

设计思路

代码如下：

```
void proc_run(struct proc_struct *proc)
{
    if (proc != current)
    {
        // LAB4:EXERCISE3
        /*
         * Some Useful MACROS, Functions and DEFINES, you can use them in below
         implementation.
         *
         * MACROS or Functions:
         *   local_intr_save():      Disable interrupts
         *   local_intr_restore():   Enable Interrupts
         *   lsatp():                Modify the value of satp register
         *   switch_to():            Context switching between two processes
         */
        bool intr_flag;
        local_intr_save(intr_flag);
        struct proc_struct *prev = current;
        current = proc;
        lsatp(current->pgdir);
        switch_to(&(prev->context), &(current->context));
        local_intr_restore(intr_flag);
    }
}
```

首先，完成中断状态的保存，调用了 `local_intr_save` 函数，并将保存的中断状态存储在 `intr_flag` 变量中。

然后，就开始做上下文切换。通过 `struct proc_struct *prev = current` 保存当前正在运行的进程结构体指针到 `prev` 变量中。在将 `current` 指针更新为要运行的进程 `proc`，这表示从现在起，系统将认为 `proc` 是当前正在运行的进程，接着切换页表 `lsatp(current->pgdir)`，重新加载 `satp` 寄存器的值为要切换到的进程（线程）的页目录表的基址，完成进程间的页表切换。

接着，调用 `switch_to` 函数，传入当前进程（`prev`）的上下文结构体指针和要运行的进程（`proc`）的上下文结构体指针。

最后在完成进程切换相关的核心操作后，通过调用 `local_intr_restore` 函数，并传入之前保存的中断状态 `intr_flag`，来恢复系统的中断状态。

问题回答

总共创建了两个内核线程，分别为：

- `idle_proc`，为第 0 个内核线程，在一开始时 `proc_init` 使用，在完成新的内核线程的创建以及各种初始化工作之后，进入死循环，用于调度其他进程或线程。
- `init_proc`，被创建用于打印 "Hello World" 的线程，在 `kernel_thread(init_main, ...)` 调度后使用，是本次实验的内核线程，只用来打印目标字符串。

扩展练习 Challenge:

1、说明语句

`local_intr_save(intr_flag); ... local_intr_restore(intr_flag);` 是如何实现开关中断的?

回答

两个函数在 `kern/sync/sync.h` 中的定义如下:

```
#ifndef __KERN_SYNC_SYNC_H__
#define __KERN_SYNC_SYNC_H__

#include <defs.h>
#include <intr.h>
#include <riscv.h>

static inline bool __intr_save(void) {
    if (read_csr(sstatus) & SSTATUS_SIE) {
        intr_disable();
        return 1;
    }
    return 0;
}

static inline void __intr_restore(bool flag) {
    if (flag) {
        intr_enable();
    }
}

#define local_intr_save(x) \
    do {                    \
        x = __intr_save(); \
    } while (0)
#define local_intr_restore(x) __intr_restore(x);

#endif /* !__KERN_SYNC_SYNC_H__ */
```

当调用 `local_intr_save` 时, 会读取 `sstatus` 寄存器, 判断 `SIE` 位的值, 如果该位为1, 则说明中断是能进行的, 这时需要调用 `intr_disable` 将该位置0, 并返回1, 将 `intr_flag` 赋值为1; 如果该位为0, 则说明中断此时已经不能进行, 则返回0, 将 `intr_flag` 赋值为0。这样就可以保证之后的代码执行时不会发生中断。

当需要恢复中断时, 调用 `local_intr_restore`, 需要判断 `intr_flag` 的值, 如果其值为1, 则需要调用 `intr_enable` 将 `sstatus` 寄存器的 `SIE` 位置1, 否则该位依然保持0。以此来恢复调用 `local_intr_save` 之前的 `SIE` 的值。

2、深入理解不同分页模式的工作原理（思考题）

`get_pte()` 函数（位于 `kern/mm/pmm.c`）用于在页表中查找或创建页表项，从而实现对指定线性地址对应的物理页的访问和映射操作。这在操作系统中的分页机制下，是实现虚拟内存与物理内存之间映射关系非常重要的内容。

- `get_pte()` 函数中有两段形式类似的代码，结合 `sv32`, `sv39`, `sv48` 的异同，解释这两段代码为什么如此相像。
- 目前 `get_pte()` 函数将页表项的查找和页表项的分配合并在一个函数里，你认为这种写法好吗？有没有必要把两个功能拆开？

回答

为什么 `get_pte()` 里两段代码“长得很像”

`get_pte()` 的定义如下：

```

//寻找(有必要的时候分配)一个页表项
pte_t *get_pte(pte_t *pgdir, uintptr_t la, bool create)
{
    pte_t *pdep1 = &pgdir[PDX1(la)]; //找到对应的Giga Page
    if (!(*pdep1 & PTE_V)) //如果下一级页表不存在，那就给它分配一页，创造新页表
    {
        struct Page *page;
        if (!create || (page = alloc_page()) == NULL)
        {
            return NULL;
        }
        set_page_ref(page, 1);
        uintptr_t pa = page2pa(page);
        memset(KADDR(pa), 0, PGSIZE);
        //我们现在在虚拟地址空间中，所以要转化为KADDR再memset.
        //不管页表怎么构造，我们确保物理地址和虚拟地址的偏移量始终相同，
        //那么就可以用这种方式完成对物理内存的访问。
        *pdep1 = pte_create(page2ppn(page), PTE_U | PTE_V); //注意这里R,W,X全零
    }
    pte_t *pdep0 = &((pte_t *)KADDR(PDE_ADDR(*pdep1))[PDX0(la)]; //再下一级页表
    //这里的逻辑和前面完全一致，页表不存在就现在分配一个
    if (!(*pdep0 & PTE_V))
    {
        struct Page *page;
        if (!create || (page = alloc_page()) == NULL)
        {
            return NULL;
        }
        set_page_ref(page, 1);
        uintptr_t pa = page2pa(page);
        memset(KADDR(pa), 0, PGSIZE);
        *pdep0 = pte_create(page2ppn(page), PTE_U | PTE_V);
    }
}
```

```
//找到输入的虚拟地址1a对应的页表项的地址(可能是刚刚分配的)
return &((pte_t *)KADDR(PDE_ADDR(*pdep0)))[PTX(1a)];
}
```

相似的原因有两点：

- **多级页表的结构相同、操作重复：**

- RISC-V 的分页 (sv32/sv39/sv48) 都是“多级页表递归索引 → 若不存在则分配下一层页表 → 继续下一级”的模式。
- 在我们的 `get_pte()` 中：
 - 第一段处理顶层表项（相当于 L2, `PDX1(1a)`），若无效则“分配一页并清零，写入页表项为有效（仅 V/U 权限）”，再进入下一层。
 - 第二段处理次顶层表项（相当于 L1, `PDX0(1a)`），逻辑与第一段完全一样。
 - 最后返回的是第三级（L0, `PTX(1a)`）中的最终 PTE 地址。
- 这就是“层层相似”的原因：每一层都是“索引 → 判断有效 → 必要时分配清零 → 前进”的同构操作，只有索引位宽与层数不同。

- **和 sv32/sv39/sv48 的对应关系：**

- sv32：2 级页表（L1→L0）。实现上只需要一段“如果无效就分配下一层”的代码块，然后返回 L0 的 PTE。
- sv39：3 级页表（L2→L1→L0）。需要两段“如果无效就分配”的相似代码（正如你当前的实现），再返回 L0 的 PTE。
- sv48：4 级页表（L3→L2→L1→L0）。会出现三段相似代码。
- 因此，层数越多，就会多出一段几乎相同的“走一层、必要时分配”逻辑，这是正常现象。

查找与分配是否应合并在一个函数里

- **现在的写法优点（带 create 标志位）：**

- 接口简单：调用者通过 `create` 控制是否需要“边查找边创建”，常用场景下省去重复代码。
- 性能友好：只遍历一遍层级，避免“先查找失败再单独创建再查找一次”的双遍历。
- 语义清晰：需要保证存在时传 `create=true`，只读路径传 `create=false`，避免误创建。

- **可能的不足：**

- 职责略混合：函数既做查找又做修改（分配/写表），从“单一职责”角度略显杂糅。
- 可测试性：把“只查找”和“创建路径”的逻辑拆分，有助于分别单测、审计权限与错误处理。

- **是否有必要拆分：**

- 视代码规模与复用需求而定。若后续支持多模式（sv32/sv39/sv48）且层数可配置，推荐抽象成：
 - 只读的 `walk_pte(pgdir, 1a)`（不创建）
 - “确保存在”的 `ensure_pte(pgdir, 1a)`（必要时创建）
 - 或者写成一个通用的“多级表遍历器”，传入“是否在缺失时创建”的策略参数。

对当前教学/实验代码体量而言，“一个函数 + create 标志位”已经足够清晰高效，不强制需要拆。

实验结果

实验结果如下图所示：

```
rowin@Rowin:~/Documents/oslab/lab4$ make qemu
PMP0: 0x0000000080000000-0x000000008001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffffffff (A,R,W,X)
DTB Init
HartID: 0
DTB Address: 0x82200000
Physical Memory from DTB:
  Base: 0x0000000080000000
  Size: 0x0000000080000000 (128 MB)
  End: 0x0000000087ffffff
DTB init completed
(THU.CST) os is loading ...

Special kernel symbols:
  entry 0xc020004a (virtual)
  etext 0xc0203e2a (virtual)
  edata 0xc0209030 (virtual)
  end 0xc020d4e4 (virtual)
Kernel executable memory footprint: 54KB
memory management: default_pmm_manager
physical memory map:
  memory: 0x08000000, [0x80000000, 0x87ffffff].
vapaofset is 18446744070488326144
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmallocc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
alloc_proc() correct!
++ setup timer interrupts
this initproc, pid = 1, name = "init"
To U: "Hello world!!".
To U: "en.., Bye, Bye. :)"
kernel panic at kern/process/proc.c:395:
  process exit!!.

Welcome to the kernel debug monitor!!
Type 'help' for a list of commands.
```